

MODULE 5

Collaborating in a Community of Developers

Mathematics Instruction | 6 Instructional Hours

1 MODULE OVERVIEW

This module bridges the technical and human dimensions of open collaborative development. You will develop practical skills in Git and GitHub workflows that enable real-time collaboration on shared EdTech projects, and examine the interpersonal and interdisciplinary dynamics that determine whether a developer community succeeds or fragments. Drawing on your work in Modules 3 and 4, you will contribute meaningfully to a shared project and critically reflect on what makes collaboration productive, equitable, and sustainable.

LEARNING OUTCOMES — By the end of this module, you will be able to:

1. Use basic Git and GitHub for collaborative development.
2. Examine interdisciplinary teamwork norms in EdTech communities.
3. Contribute meaningfully to a shared EdTech project.
4. Critique collaboration challenges and successes.

2 GIT AND GITHUB FOR COLLABORATIVE DEVELOPMENT

2.1 1.1 Why Version Control is Essential for Collaboration

When a single person works on a document or codebase, changes are straightforward to track. When five, fifty, or five thousand people work on the same project simultaneously — as happens in major open-source EdTech tools — the coordination problem becomes complex. **Version control** solves this problem by:

- Recording every change ever made, who made it, when, and why.
- Allowing multiple people to work on different parts of the project at the same time without overwriting each other's work.
- Enabling any past state of the project to be restored if a change introduces a problem.
- Providing a transparent, auditable record of how the project evolved.

Git is the dominant version control system in software and EdTech development. **GitHub** is the most widely used platform for hosting Git repositories and managing collaborative workflows. Together, they are the infrastructure of modern open-source collaboration.

2.2 1.2 Core Git Concepts: A Structured Reference

Repository	The complete project: all files, all history, all branches. Stored locally and/or on GitHub.
Commit	A saved snapshot of changes, accompanied by a message. The atomic unit of version control history.
Branch	A parallel line of development. Changes on a branch do not affect the main branch until deliberately merged.
Merge	Combining changes from one branch into another. May require resolving conflicts if both branches changed the same content.
Conflict	Occurs when two branches have made incompatible changes to the same part of a file. Must be resolved manually before merging.
Remote	A version of the repository hosted on a server (e.g., GitHub). <code>origin</code> refers to your fork; <code>upstream</code> refers to the original project.
Push	Sending local commits to a remote repository.
Pull / Fetch	Retrieving changes from a remote repository. <code>fetch</code> downloads without applying; <code>pull</code> downloads and merges.
Rebase	Replaying your branch's commits on top of another branch's latest state, producing a cleaner linear history.
Stash	Temporarily storing uncommitted changes so you can switch branches without losing work in progress.
Tag	A named reference to a specific commit, typically used to mark release versions (e.g., <code>v1.0.0</code>).
.gitignore	A file listing paths and patterns that Git should not track (e.g., temporary files, API keys, build artefacts).

2.3 1.3 The Collaborative Git Workflow: Step by Step

The following workflow is the standard model used in collaborative open-source EdTech projects. It builds on the contribution workflow introduced in Module 3 with a deeper focus on the mechanics of collaboration.

2.3.1 Setting Up the Collaborative Environment

Listing 1: Initial repository setup for a collaborator

```

1 # Clone the shared project repository
2 git clone https://github.com/edtech-org/mathtools.git
3 cd mathtools
4
5 # Add the upstream remote (the original project, not your fork)
```

```

6 git remote add upstream
   https://github.com/edtech-org/mathtools.git
7
8 # Verify your remotes
9 git remote -v
10 # origin      https://github.com/YOUR-USERNAME/mathtools.git (fetch)
11 # origin      https://github.com/YOUR-USERNAME/mathtools.git (push)
12 # upstream    https://github.com/edtech-org/mathtools.git (fetch)
13 # upstream    https://github.com/edtech-org/mathtools.git (push)

```

2.3.2 Daily Workflow: Stay Synchronised

Listing 2: Keeping your local copy current with the shared project

```

1 # Switch to main branch
2 git checkout main
3
4 # Fetch the latest changes from the original project
5 git fetch upstream
6
7 # Merge upstream changes into your local main
8 git merge upstream/main
9
10 # Push the updated main to your fork
11 git push origin main

```

2.3.3 Working on a Contribution

Listing 3: Creating and working on a feature branch

```

1 # Create and switch to a new branch for your contribution
2 git checkout -b add-grade7-ratio-activity
3
4 # ... make your changes to files ...
5
6 # Check what has changed
7 git status
8 git diff
9
10 # Stage specific files (preferred over staging everything)
11 git add activities/grade7/ratio_introduction.json
12 git add docs/grade7_teacher_guide.md
13
14 # Commit with a clear, informative message
15 git commit -m "Add Grade 7 ratio introduction activity"
16
17 - Three-part activity: concrete -> pictorial -> abstract
18 - Aligned to CAPS Grade 7 Number strand, Section 1.3
19 - Includes teacher guide with worked solutions
20 - Uses local Kenyan market context for word problems
21

```

```

22 Closes #147"
23
24 # Push to your fork
25 git push origin add-grade7-ratio-activity

```

2.3.4 Resolving a Merge Conflict

Merge conflicts occur when two collaborators modify the same lines of a file. They must be resolved manually.

Listing 4: Identifying and resolving a merge conflict

```

1 # Attempt to merge upstream changes
2 git merge upstream/main
3
4 # Git reports a conflict:
5 # CONFLICT (content): Merge conflict in
6   activities/grade6/fractions.json
7
8 # Open the conflicted file - Git marks the conflict:
9 # <<<<<< HEAD
10 # "difficulty": "intermediate",
11 # =====
12 # "difficulty": "beginner",
13 # >>>>>> upstream/main
14
15 # Decide which version is correct (or combine both)
16 # Remove the conflict markers and save the file
17
18 # Mark as resolved and complete the merge
19 git add activities/grade6/fractions.json
20 git commit -m "Resolve merge conflict: keep intermediate
21   difficulty for fractions activity"

```

CONFLICT PREVENTION STRATEGIES

Most merge conflicts can be prevented through good collaboration habits:

- Synchronise with upstream *daily* — the longer you work without syncing, the more conflicts accumulate.
- Keep branches short-lived — complete contributions quickly; do not maintain long-running branches.
- Communicate before starting work that overlaps with a colleague's area.
- Break large contributions into small, focused pull requests.

2.4 1.4 GitHub Collaboration Features

GitHub provides features beyond basic Git that are essential for team collaboration:

Issues	The task management system. Log bugs, feature requests, content gaps, and discussion threads. Use labels (bug, enhancement, documentation, good-first-issue) to categorise and route work.
Pull Requests	Proposals to merge a branch into the main codebase. The primary site of peer review, discussion, and approval.
Projects (Kanban)	Visual project boards that track issues and PRs through workflow stages (Backlog, In Progress, In Review, Done).
Milestones	Group related issues and PRs into a named goal with a deadline (e.g., “v2.0 Release”, “CAPS Alignment Sprint”).
Discussions	A forum for open-ended conversations that do not fit the issue format: design decisions, community announcements, Q&A.
Wiki	Project documentation hosted within the repository. Used for contributor onboarding guides, architecture notes, and conventions.
Actions (CI/CD)	Automated workflows triggered by repository events (e.g., automatically run tests when a PR is opened; auto-deploy documentation on merge).
Code Review Tools	Inline comments on specific lines, suggested changes, and approve/request-changes decisions within PRs.

2.5 1.5 Writing Effective Commit Messages

A commit message is a communication to every future collaborator (including your future self). The widely adopted convention is:

COMMIT MESSAGE CONVENTION

Format:

```
<type>(<scope>): <short summary in imperative mood> (50 chars max)
```

```
<blank line>
```

```
<body: explain what and why, not how> (72 chars per line)
```

```
<footer: link issues, breaking changes>
```

Common types: feat (new feature), fix (bug fix), docs (documentation), style (formatting), refactor, test, chore (maintenance)

Examples:

```
feat(grade8): add linear equations worked examples
```

```
fix(accessibility): correct colour contrast in feedback
```

```

widget
docs(contributing): clarify PR template instructions

```

3 INTERDISCIPLINARY TEAMWORK NORMS

3.1 2.1 Who is in an EdTech Development Community?

Open-source mathematics EdTech communities are inherently **interdisciplinary** — they bring together people with fundamentally different training, vocabulary, and professional instincts. Understanding the perspectives and working styles of other disciplines is a prerequisite for effective collaboration.

Mathematics Educator	Subject knowledge, pedagogy, curriculum	“Is this mathematically correct and pedagogically sound? Will it work in a real classroom?”
Software Developer	Code architecture, performance, maintainability	“Is this code clean, testable, and efficient? Does it follow our conventions?”
UX / Interaction Designer	Usability, user research, visual design	“Is this interface intuitive, accessible, and aesthetically clear?”
Curriculum Specialist	Curriculum frameworks, standards alignment	“Does this align with the target curriculum? Is the scope and sequence correct?”
Accessibility Expert	WCAG, assistive technologies, inclusive design	“Who is excluded by this design? What barriers need to be removed?”
Translator / Localiser	Language, culture, contextualisation	“Does this make sense in our local context? Are cultural assumptions embedded?”
Community Manager	Communication, onboarding, contributor relations	“Are contributors welcomed, supported, and retained? Is the community healthy?”
Learner / Student	Lived experience as the end user	“Does this actually make sense to me? Is this what I need?”

3.2 2.2 Vocabulary Gaps and the Communication Problem

The single most persistent challenge in interdisciplinary EdTech teams is **vocabulary mismatch**: the same word means different things in different professional contexts, and each discipline has jargon that is opaque to others.

“Scaffold”	A structured support that gradually reduces as learners develop independence.	A temporary code framework or boilerplate used during development.
“Deploy”	Distribute materials to students or teachers.	Push code to a live server where users can access it.
“Test”	An assessment given to students to measure learning.	Automated code that verifies software functions correctly.
“Fork”	Splitting a lesson or content path for different learner groups.	Creating a personal copy of a repository.
“Issue”	A problem a student is experiencing.	A formal task item in the project tracker.
“Merge”	Combining student groups or activity streams.	Integrating changes from one branch into another.

STRATEGY: BUILD A SHARED GLOSSARY

In the early stages of an interdisciplinary project, invest time in building a shared project glossary — a simple document (often a GitHub Wiki page) that defines key terms as they are used *in this project*. When vocabulary disputes arise (and they will), the glossary provides a neutral reference point. Each discipline contributes their own terms; each team member reviews terms from other disciplines. This is not a one-time exercise: the glossary grows throughout the project.

3.3 2.3 Norms for Effective Interdisciplinary Collaboration

The following norms are consistently associated with high-performing interdisciplinary EdTech teams:

1. **Assume good intent.** When a developer questions a pedagogical decision or an educator pushes back on a technical constraint, assume the other person is trying to serve the learner — not undermining your expertise.
2. **Explain the reasoning, not just the decision.** “We need three hint levels” is a decision. “Research on worked examples shows that fading support in three stages reduces cognitive load without removing scaffolding prematurely” is the reasoning. Sharing the reasoning enables the other discipline to evaluate it — and sometimes to suggest a better solution.
3. **Separate the role from the person.** “This design will confuse learners” is feedback about a design decision. “You don’t understand learners” is an attack on a person. The distinction matters enormously for psychological safety in a team.

4. **Make decisions by expertise, not seniority.** On mathematical accuracy questions, the mathematician’s assessment takes precedence. On accessibility questions, the accessibility expert’s view is primary. On code architecture, the lead developer decides. Explicitly agreeing on decision authority in each domain prevents endless debate.
5. **Document decisions.** When a design choice is made after discussion, record the decision and its rationale in the project’s documentation (issue comment, meeting notes, ADR — Architecture Decision Record). This prevents the same argument from recurring when team members change.
6. **Celebrate different contributions equally.** A well-written translation is as valuable as a well-written function. A curriculum alignment document is as important as a user interface improvement. Teams that visibly value all contribution types retain contributors from all disciplines.
7. **Handle conflict through the work.** When interpersonal tensions arise, redirect the conversation to the specific design or content question at issue. Ask: “What outcome is best for the learner?” This shared reference point often resolves disagreements that personal differences cannot.

3.4 2.4 Psychological Safety in Developer Communities

Psychological safety — the shared belief that team members can speak up, take risks, and make mistakes without fear of humiliation or punishment — is the single most consistent predictor of high-performing teams (Edmondson, 1999; Google Project Aristotle, 2016).

In open-source communities, psychological safety has a specific character:

- **Asking “stupid” questions is normal and valued.** Every expert was once a beginner. Communities that mock novice questions rapidly lose new contributors.
- **Being wrong is recoverable.** Git means every mistake can be reverted. A wrong commit can be fixed. A faulty PR can be closed. No error is permanent.
- **Disagree publicly and respectfully.** Open communities conduct their disagreements in public issue threads — this is a feature, not a bug. Disagreements conducted respectfully produce better decisions and build community trust.
- **Failure is data.** When a feature does not work, or a localisation is rejected by users, this information improves the next iteration. Treating failure as information rather than blame is essential to iterative improvement.

4 CONTRIBUTING MEANINGFULLY TO A SHARED EDTECH PROJECT

4.1 3.1 What Makes a Contribution “Meaningful”?

Not all contributions have equal impact. A meaningful contribution is one that:

- Addresses a real, documented need in the project (linked to an open issue or agreed roadmap item).

- Meets the project's quality standards (follows contribution guidelines, passes review).
- Is sustainable — it does not create technical debt, inconsistency, or maintenance burden that outweighs its value.
- Respects the project's vision and does not unilaterally redirect the project's scope.
- Is documented well enough that future contributors can understand and build upon it.

4.2 3.2 Identifying and Scoping Your Contribution

New contributors most commonly fail not because their work is poor quality, but because they attempt contributions that are **too large, too vague, or unsolicited**. The following scoping principles apply:

CONTRIBUTION SCOPING PRINCIPLES

- **Start small:** Your first contribution to a project should be small enough to be reviewed and merged within a week. This builds trust, familiarises you with project norms, and creates a positive working relationship with maintainers.
- **One thing per PR:** A pull request should address one issue or one clearly defined improvement. Mixed-purpose PRs are harder to review, harder to revert, and slower to merge.
- **Solve a stated problem:** Contributing solutions to undocumented problems that only you perceive often leads to rejection. Check that your contribution addresses a need the community has already recognised.
- **Check for overlap:** Before starting, search existing issues and open PRs to ensure no one is already working on the same thing.
- **Discuss first for large contributions:** If you want to make a substantial change — a new feature, a curriculum redesign, a major localisation — open a discussion issue first and get maintainer agreement before investing weeks of effort.

4.3 3.3 The RACI Model for Collaborative Roles

The **RACI model** is a widely used framework for clarifying who does what in collaborative projects:

R	Responsible	The person who does the work. For a localisation contribution: the translator.
A	Accountable	The person who owns the outcome and approves the work. For a localisation: the project maintainer.
C	Consulted	People whose input is sought before a decision. For a localisation: a curriculum specialist and a native speaker from the target region.
I	Informed	People who are notified of progress but do not input. For a localisation: the broader contributor community.

Explicit RACI agreements prevent two of the most common collaboration failures: **duplication of effort** (when two people both think they are Responsible for the same task) and **decision paralysis** (when no one is clearly Accountable).

4.4 3.4 Asynchronous Communication Norms

Open-source EdTech communities are **asynchronous by default**: contributors live in different time zones, have different availability windows, and communicate through text rather than real-time conversation. This requires deliberate communication norms:

- **Be explicit and complete in written communication.** In a real-time conversation, you can ask for clarification immediately. In an issue comment or PR review, an ambiguous message may not be resolved for 24 hours. Write as if the reader cannot ask a follow-up question.
- **Use threads, not messages.** GitHub Issues and Discussions support threaded replies. Use them to keep discussions organised rather than creating a stream of disconnected comments.
- **Reference everything.** When referring to an issue in a comment, use the `#issue-number` syntax so GitHub creates a hyperlink automatically. When referring to a specific commit or file, link directly to it.
- **Signal your availability and timeline.** If you have taken on an issue but will need two weeks to complete it, say so. Silence is interpreted as abandonment in async communities.
- **Mark stale discussions.** If a thread has been inactive for a significant period, check in: is this issue still relevant? Has someone already solved it elsewhere?

4.5 3.5 Case Study: Interdisciplinary Team Contributing to Kolibri

Context: A team of four professionals — a secondary mathematics teacher, a front-end developer, a Swahili translator, and a curriculum alignment specialist — formed a team to contribute a Grade 6 fractions content channel to Kolibri for deployment in Tanzanian schools.

Team setup:

- They created a shared GitHub project board with four columns: Backlog, In Progress, In Review, Done.
- They wrote a team agreement in the repository Wiki covering: communication channel (GitHub Discussions for async; weekly video call for decisions), decision authority (educator on pedagogy; developer on technical; consensus on scope), and contribution standards.
- They opened a discussion issue on the Kolibri GitHub repository to introduce themselves and confirm their approach with maintainers before starting work.

Collaboration challenges encountered:

- The developer's first implementation of the fraction tool used colour as the sole differentiator — identified by the educator and fixed before any review by maintainers.
- The translator's Swahili content was initially based on Tanzanian Standard Swahili; the curriculum specialist noted that several terms differed from the phrasing used in Tanzanian school textbooks and requested revisions.
- Two team members worked on the same JSON configuration file simultaneously without creating separate branches, resulting in a merge conflict that took 90 minutes to resolve.

Collaboration successes:

- The educator's knowledge of learner misconceptions improved the scaffolding design significantly compared to the developer's initial approach.
- The curriculum specialist's alignment document became the most-bookmarked resource in the project.
- After the merge conflict incident, the team adopted a file ownership convention: each contributor owned specific folders and coordinated before touching shared configuration files.

Outcome: The content channel was merged after two rounds of review. It has since been downloaded and deployed in 23 Tanzanian schools. The team's collaboration agreement and file ownership convention were adopted as recommended practice in the Kolibri contributor documentation.

5 CRITIQUING COLLABORATION: CHALLENGES AND SUCCESSES

5.1 4.1 A Framework for Critiquing Collaboration

Productive critique of collaboration requires a structured framework. Simply listing “what went wrong” and “what went right” is insufficient: a useful critique identifies the *root causes* of problems, the conditions that enabled successes, and *transferable lessons* for future collaborations.

The following four-lens framework structures a thorough collaboration critique:

Process	Did the workflow support or hinder the work?	Branch strategy, PR size, sync frequency, communication timeliness, role clarity.
People	Did team dynamics support or undermine quality?	Psychological safety, conflict handling, power dynamics, workload equity, recognition.
Product	Did the collaboration produce something of genuine value?	Quality of output, alignment with stated goals, user adoption, maintainability.
Learning	Did participants grow through the collaboration?	New skills acquired, misconceptions corrected, professional networks built, lessons documented.

5.2 4.2 Common Collaboration Failures: Root Cause Analysis

5.2.1 Failure: The Silent Contributor

Symptom: A team member takes on an issue, makes no visible progress for several weeks, and does not respond to check-ins. The issue is eventually reassigned.

Root causes:

- The task was underestimated at scoping and was larger than the contributor could complete in available time.
- The contributor encountered a technical or content problem they did not know how to resolve and felt unable to ask for help (low psychological safety).
- The contributor's circumstances changed and they could not communicate this without feeling they had "failed."

Systemic fix: Normalise partial contributions and check-ins. A norm of "update the issue by Friday, even if the update is: I'm stuck on X" removes the fear of silence. Build in explicit "help-wanted" signals (help-wanted label on issues) so contributors can ask for assistance without shame.

5.2.2 Failure: The Rejected Contribution

Symptom: A contributor invests significant time in a large pull request. After weeks of waiting, a maintainer closes it with a brief explanation. The contributor leaves the community.

Root causes:

- The contribution addressed a problem the core team had already decided not to prioritise.
- The contribution conflicted with an architectural decision the contributor was not aware of.
- The contribution was too large to review efficiently and maintainers kept deferring it.

Systemic fix: Require maintainer sign-off on the approach before large contributions begin. Provide a `CONTRIBUTING.md` that explicitly describes contribution size expectations. When closing a PR, always acknowledge the work invested and explain the decision respectfully.

5.2.3 Failure: The Discipline Silo

Symptom: Educators and developers work in parallel without genuine integration. The tool is technically functional but pedagogically weak; or pedagogically well-designed but technically unmaintainable.

Root causes:

- Roles were defined too narrowly: educators reviewed content; developers reviewed code. Neither reviewed the integrated experience.
- Reviews were sequential rather than collaborative: developers implemented what educators specified without discussion.
- No shared ownership of the user experience existed.

Systemic fix: Cross-functional reviews. Require at least one educator and one developer to review every PR that involves both content and implementation. Schedule regular integration reviews where the full team evaluates the tool as a learner would experience it.

5.2.4 Failure: The Burnout Spiral

Symptom: A small number of highly committed contributors do an increasing share of the work. Over time, they become exhausted, resentful, or simply leave. The project stalls.

Root causes:

- The bus factor is too low: one or two people hold critical knowledge that is not documented.
- The project lacks onboarding infrastructure, so new contributors require significant support from existing contributors to get started.
- Contributions are not equally visible: code contributions are counted; translation and documentation are not.

Systemic fix: Deliberately distribute knowledge through documentation, pair programming, and contributor rotation. Celebrate all contribution types visibly. Invest in onboarding infrastructure that allows new contributors to become productive with minimal support from existing contributors.

5.3 4.3 Markers of Successful Collaboration

Against the failure modes above, the following are empirically associated with successful open EdTech collaborations:

- **High bus factor:** Multiple contributors understand each major area of the project. No single person's departure would be catastrophic.

- **Low barrier to first contribution:** A new contributor can make a meaningful, merged contribution within a few hours of first engaging with the project.
- **Active issue triage:** Issues are regularly reviewed, labelled, and assigned. The issue tracker reflects the real current state of the project.
- **Responsive review:** PRs receive a first response within 48 hours. Contributors are not left in silence.
- **Documented decisions:** When the team makes a significant design or scope decision, it is recorded in a discoverable place.
- **Visible progress:** Milestones, release notes, and changelogs allow all contributors to see how the project is moving forward.
- **Welcoming culture:** New contributors are explicitly thanked. Mistakes are corrected without contempt.

6 PRACTICAL COLLABORATION ACTIVITIES

6.1 5.1 Activity 1: Shared Repository Sprint

Task: In groups of three to four, set up a shared GitHub repository for a collaborative mathematics EdTech micro-project (e.g., a set of Grade 7 algebra activities in a structured JSON format).

Each team member must:

1. Fork the shared repository and clone it locally.
2. Create a branch and make a defined contribution (one activity, one piece of documentation, or one localisation element).
3. Open a pull request with a well-structured description.
4. Review at least one other team member's PR using the constructive review principles from Module 4.
5. Resolve at least one merge conflict collaboratively.

After the sprint: Each team writes a 500-word collaboration critique using the four-lens framework (Process, People, Product, Learning).

6.2 5.2 Activity 2: Interdisciplinary Role Simulation

Task: Each team member is assigned a role (educator, developer, UX designer, translator). The team must agree on the design of a single interactive mathematics activity for Grade 5 fractions.

The design process must include:

- A 20-minute synchronous design discussion (recorded or minuted).
- A written decision record capturing the three most significant design choices and the reasoning behind each.
- Identification of at least one moment where disciplines disagreed and documentation of how the disagreement was resolved.

Debrief questions:

- What did each role prioritise that the others initially overlooked?
- What vocabulary confusion arose and how was it resolved?
- What does the final design owe to each discipline?

6.3 5.3 Activity 3: Contribution to a Live Open-Source Project

Task: Individually, make a real contribution to a live open-source mathematics EdTech project:

1. Browse open issues labelled `good-first-issue` or `help-wanted` on GeoGebra Materials, Kolibri, H5P, or a similar project.
2. Select an issue aligned with your skills (documentation, content, translation, or a simple code fix).
3. Follow the full contribution workflow: fork, branch, commit, PR.
4. Submit the PR and engage with any review feedback.

Reflection: Write a 300-word appraisal of the collaboration experience addressing: what the community norms were, how you were received as a contributor, what you learned from the process, and how the experience compared to your expectations.

7 KEY CONCEPTS AND TERMINOLOGY

Version Control	A system that records changes to files over time, enabling recall of any earlier version and coordinating contributions from multiple people.
Git	A distributed version control system. Every contributor holds a complete copy of the repository history, enabling offline work and resilience.
GitHub	A web platform for hosting Git repositories and managing collaborative workflows (issues, PRs, project boards, wikis).
Merge Conflict	A situation where two branches have made incompatible changes to the same part of a file, requiring manual resolution before merging.
Upstream	The original repository from which a fork was made. Staying synchronised with upstream ensures a contributor's fork reflects the latest project state.
Rebase	Replaying commits from one branch on top of another, producing a cleaner linear commit history.
Bus Factor	The number of contributors whose departure would cripple the project. A bus factor of 1 means the project depends critically on a single person.
Psychological Safety	The shared belief that team members can speak up, take risks, and make mistakes without fear of humiliation (Edmondson, 1999).
RACI Model	A responsibility assignment framework: Responsible (does the work), Accountable (owns the outcome), Consulted (provides input), Informed (receives updates).
Asynchronous Communication	Communication where participants do not interact in real time. The default mode in distributed open-source teams, requiring explicit, complete, and well-referenced written messages.
Interdisciplinary Team	A group combining members with different professional disciplines (e.g., mathematics education, software development, UX design) to achieve a shared goal.
Architecture Decision Record (ADR)	A short document that captures a significant design or technical decision, its context, the options considered, and the rationale for the choice made.
Kanban Board	A visual workflow management tool using columns (e.g., Backlog, In Progress, In Review, Done) to track the status of tasks. Available as GitHub Projects.
CI/CD	Continuous Integration / Continuous Deployment.
Instructional Hours: 6	Automated workflows that test and deploy code changes whenever a contribution is merged. 17
Commit Convention	An agreed format for commit messages (e.g.,

8 REVISION QUESTIONS

Use these questions to consolidate your understanding of Module 5 content:

1. Explain why version control is essential for collaborative development. A colleague argues that a shared Google Drive folder achieves the same goal. Write a response that identifies at least four specific limitations of shared cloud storage compared to Git for collaborative EdTech development.
2. Describe the difference between `git merge` and `git rebase`. In what circumstances would you choose each, and what are the risks of using rebase on a public branch?
3. You are working on a Grade 8 algebra activity in a shared repository. You run `git pull upstream main` and receive a merge conflict in `activities/grade8/algebra.json`. Describe step by step how you would resolve the conflict, including the Git commands you would use and the decision you would make about which version of the content to keep.
4. A team building an open mathematics platform includes a secondary mathematics teacher, a React developer, a UX designer, and a Swahili translator. The team is designing an adaptive fraction tool.
 - (a) Identify two vocabulary terms that could cause miscommunication between the educator and the developer.
 - (b) Describe the likely priority conflict between the UX designer and the developer on the question of accessibility.
 - (c) Propose one concrete collaboration norm that would help the team make better-integrated design decisions.
5. Using the RACI model, assign roles (R, A, C, I) for the following task to the four team members in Question 4: *“Translate all student-facing text in the fraction tool into Swahili and verify alignment with Tanzanian Grade 5 curriculum terminology.”*
6. Describe what is meant by **psychological safety** in an open-source community. Explain how the following two practices either support or undermine it:
 - (a) A maintainer closes a new contributor’s pull request with the comment: “This is not how we do things here.”
 - (b) A contributor opens an issue titled: “I’m a teacher, not a developer — can someone help me understand how to submit a content change?”
7. Identify the root cause of each of the following collaboration failures and propose a systemic fix for each:
 - (a) A contributor submits a 3,000-line pull request rewriting the entire question engine after three months of silent work. The maintainers close it without merging.
 - (b) The project’s lead developer and sole documentation author leaves the community. Within one month, new contributor onboarding effectively stops.
 - (c) Two team members both work on the same localisation file over a weekend without coordinating, resulting in conflicting changes.
8. Using the four-lens framework (Process, People, Product, Learning), critique a real or hypothetical collaboration experience of your own. Your critique should identify at least one success and one challenge under each lens and propose a transferable lesson from each.

9. Write a well-formed Git commit message (following the Conventional Commits standard) for the following change: *You have added a Grade 9 activity on solving simultaneous equations using the substitution method, aligned to the South African CAPS curriculum, using a local South African context (a tuck shop pricing problem). The commit also fixes a formatting error in an existing Grade 9 activity.*

MODULE 5 SUMMARY

- **Git and GitHub** provide the version control infrastructure for collaborative EdTech development. Core skills — branching, committing with clear messages, resolving conflicts, syncing with upstream, and opening pull requests — are learnable and essential.
- **Merge conflicts** are a normal feature of collaboration, not a failure. They are prevented by frequent synchronisation, small branches, and communication; resolved by deliberate manual decision-making.
- **Interdisciplinary teams** combine mathematics educators, developers, UX designers, translators, and curriculum specialists. Vocabulary mismatch is the most persistent communication challenge; shared glossaries, explicit reasoning, and cross-functional reviews address it.
- **Psychological safety** is a prerequisite for productive open communities. It is built through welcoming norms, Code of Conduct enforcement, treating failure as information, and modelling respectful disagreement.
- **Meaningful contributions** are small, focused, documented, linked to stated needs, and preceded by discussion for large changes. The RACI model clarifies who is responsible, accountable, consulted, and informed on each contribution.
- **Asynchronous communication** requires explicit, complete, well-referenced messages. Silence is misread as abandonment in distributed communities.
- **Collaboration failures** — the silent contributor, the rejected contribution, the discipline silo, the burnout spiral — have identifiable root causes and systemic fixes. Critiquing collaboration using the Process, People, Product, and Learning lenses produces actionable improvements.
- *The quality of a collaborative EdTech project is determined as much by how people work together as by the technical or pedagogical skills they bring individually.*

9 FURTHER READING AND RESOURCES

- **Pro Git (Chacon & Straub, 2nd ed.):** <https://git-scm.com/book/en/v2> — The definitive free, open-source guide to Git, from basics to advanced workflows.
- **GitHub Skills:** <https://skills.github.com> — Free, interactive courses covering Git, GitHub Actions, collaboration workflows, and project management.
- **Conventional Commits Specification:** <https://www.conventionalcommits.org> — The widely adopted standard for structured commit messages.
- **Edmondson, A. C. (1999). Psychological Safety and Learning Behavior in Work Teams.** *Administrative Science Quarterly*, 44(2), 350–383 — The foundational research on psychological safety in teams.

- **Google Project Aristotle (2016):** <https://rework.withgoogle.com/print/guides/5721312655835136/> — Google’s research identifying psychological safety as the primary predictor of team effectiveness.
- **Open Source Guides — Building Welcoming Communities:** <https://opensource.guide/building-community/> — Practical guidance for community managers and project maintainers.
- **Kolibri Contributor Documentation:** <https://kolibri-dev.readthedocs.io/en/develop/contributing/> — A well-structured real-world example of contributor onboarding documentation.
- **Architecture Decision Records (ADRs):** <https://adr.github.io> — Resources on the ADR format for documenting collaborative design decisions.
- **Nayebi, M. et al. (2019). Towards Actionable Analytics for Open Source Projects.** — Research on contributor behaviour patterns and community health metrics in open-source projects.